Supplementary Material for CLINES

Contents

1 Overview **2**

2 Prompt Suite **2**
2.1 Clinical Entity Extraction 2
2.2 Entity Relationship Linking 3
2.3 Entity Normalization 4
2.4 Assertion Status Classification 5
2.5 Attribute Enrichment 6
2.6 Basic Patient and Visit Information 8
2.7 Per-Chunk Date Extraction (Single-chunk Context) 9
2.8 Cross-Chunk Date Extraction (Multi-chunk Context) 9
2.9 Admission and Discharge Date Finder 10
2.10 Date Normalization (Helper within Step 3) 10
2.11 JSON Repair (Robustness Helper) 11

3 JSON Output Specification **11**

4 i2b2 Export Schema **12**
4.1 Field Definitions 12
4.2 Base and Modifier Records 13

5 UMLS Retrieval Procedure **14**

6 Mapping UMLS Concepts to Source Terminologies **15**

7 Detailed Performance Metrics with Bootstrap Confidence Intervals **15**

8 Inter-Annotator Agreement: Per-Note Detail **16**

9 Hallucination Taxonomy and Case Studies **17**
9.1 Representative case studies 18

10 Output Consistency Across Repeated Runs **19**
10.1 Protocol 19
10.2 Results 20

11 Reporting Checklists: MI-CLAIM and TRIPOD+AI 20

1. Overview

This supplementary document expands the methodological details for CLINES. We provide the complete prompting strategy, clarify the structured outputs in both JSON and i2b2 formats, and outline the retrieval procedure used for mapping clinical mentions to UMLS concepts. Terminology and task framing match the main manuscript to avoid discrepancies.

2. Prompt Suite

All prompts are delivered verbatim to the instruction-finetuned models. Listings wrap lines automatically to prevent overflow. The few-shot examples reproduced below are the exact in-context demonstrations embedded in the deployed prompt templates (the `{note}` placeholder marks where the input note is inserted at run time). They are reproduced here in full for reproducibility.

2.1. Clinical Entity Extraction

Purpose. Detects problems, findings, procedures, medications, tests, and other biomedical entities; values for panel tests are separate entities.

````
## Extract biomedical entities from the electronic health record below. Include
     tests, procedures, symptoms, diseases, allergies, medications, etc. Mark
     each entity with <KEY> tags (e.g., <1>entity</1>, <2>entity</2>).

## Rules:
- Generally DO NOT mark values/units/routes/frequencies associated with primary
     entities
- EXCEPTION: For panel tests (Chem-7, CMP, CBC), mark each value as separate
     entity
- Include the main entity name even with abbreviations
- For overlapping entities, prioritize the most specific concept
- Mark negated findings (e.g., "no fever" -> mark "fever")

## Examples:

Input:
```
The patient was taken to the Operating Room for wound exploration directly from
     the Trauma Room. TUMOR SIZE: 4 x 3.2 x 3 cm, Chem-7: 127, 3.6, 88, 29, 16.5,
      0.6, 143.
```

Output:
```
The patient was taken to the <1>Operating Room</1> for <2>wound exploration</2>
     directly from the <3>Trauma Room</3>. <4>TUMOR SIZE</4>: 4 x 3.2 x 3 cm, <5>
     Chem-7</5>: <6>127</6>, <7>3.6</7>, <8>88</8>, <9>29</9>, <10>16.5</10>,
     <11>0.6</11>, <12>143</12>.
```
````

````
Input:
```
Hydrocodone 7.5mg + Apap 325mg 7.5-325MG take 1 PO PRN arthritis; Pt c/o SOB and
     CP. EKG showed NSR. CBC with WNL RBC but WBC 12.4.
```

Output:
```
<1>Hydrocodone</1> 7.5mg + <2>Apap</2> 325mg 7.5-325MG take 1 PO PRN <3>arthritis
     </3>; <4>Pt</4> <5>c/o</5> <6>SOB</6> and <7>CP</7>. <8>EKG</8> showed <9>
     NSR</9>. <10>CBC</10> with <11>WNL</11> <12>RBC</12> but <13>WBC</13> 12.4.
```

Input:
```
{note}
```

Please respond with only the annotated record. Do not include any additional text
     . Output:
````

2.2. Entity Relationship Linking

Purpose. Links nearby entities with directed clinical relations (treatment, causal, measurement, anatomical, temporal, or general relatedness).

```
## Identify relationships between marked entities in this health record. Find
     which entities are clinically related to each other and determine their
     relationship type.

## Output a JSON list of dictionaries with:
- tag: entity order number (from KEY tags)
- related: a dictionary where keys are tags of related entities and values are
     relationship types FROM the perspective of the entity with the "tag" key TO
     the entity in the "related" dictionary keys

## IMPORTANT RULES:
1. Relationship Direction:
   - The relationship is always FROM the entity with the "tag" field TO the
       entity in the "related" field keys
   - For example: {"tag": "1", "related": {"2": "treated_by"}} means "Entity 1 is
        treated by Entity 2"
   - Similarly: {"tag": "2", "related": {"1": "treats"}} means "Entity 2 treats
       Entity 1"
2. Proximity:
   - Only identify relationships between entities that are close to each other in
        the text
   - Focus on entities within the same sentence or adjacent sentences
   - Do not try to link entities that are far apart (e.g., different paragraphs)
   - Prioritize obvious, direct relationships over tenuous connections

## Relationship types include (non-exhaustive):
- Treatment: treats / treated_by / manages / managed_by / alleviates /
     alleviated_by
- Causal: causes / caused_by / exacerbates / exacerbated_by / indicates /
     indicated_by / risk_factor / at_risk_from / complication_of /
     has_complication
```

```
- Measurement: measures / measured_by / evaluates / evaluated_by / diagnoses /
    diagnosed_by / monitors / monitored_by / has_value / value_of / has_unit /
    unit_for
- Anatomical: located_in / location_of / part_of / has_part / administered_at /
    site_for
- Temporal: precedes / follows / concurrent_with
- Clinical: symptom_of / has_symptom / finding_of / has_finding /
    manifestation_of / has_manifestation / has_dosage / dosage_for /
    has_frequency / frequency_for / has_route / route_for / component_of /
    has_component
- Default: related_to

## Examples:

Input:
```
Patient with <1>hypertension</1> takes <2>lisinopril</2> 20mg daily. <3>Blood
    pressure</3> was <4>142/88</4> <5>mmHg</5>.
```

Output:
```
[
  {"tag": "1", "related": {"2": "treated_by", "3": "measured_by"}},
  {"tag": "2", "related": {"1": "treats"}},
  {"tag": "3", "related": {"1": "measures", "4": "has_value", "5": "has_unit"}},
  {"tag": "4", "related": {"3": "value_of", "5": "measured_in"}},
  {"tag": "5", "related": {"3": "unit_of", "4": "unit_for"}}
]
```

Input:
```
{note}
```

## Please respond with valid JSON only, no additional text. Output:
```

2.3. Entity Normalization

Purpose. Converts abbreviations and informal mentions to standardized biomedical terms for ontology mapping.

```
## Recover standard names/synonyms for marked entities in this health record. For
     abbreviations, provide full forms. For values (numbers/units), infer the
    biomedical concept.

## Output a JSON list of dictionaries with:
- TAG: the order number of the entity (from KEY tags)
- CLEAN: standardized name/synonym for the entity

## Handling ambiguity:
- Use context to disambiguate abbreviations
- For unclear abbreviations, provide most common medical expansion
- For values, infer relevant biomedical concepts
- Standardize vague symptoms to specific medical terms
```

```
## Example:
Input:
```
The patient reported <1>HTN</1> history and underwent <2>BP Measurement</2>,
     which showed <3>150/90 mmHg</3>. Labs revealed <4>Elevated A1C</4> at
     <5>7.5%</5>. Follow-up tests ruled out <6>CAD</6>, but a <7>1.5 cm Mass</7>
     was detected.
```

Output:
```
[
  {"TAG": "1", "CLEAN": "Hypertension"},
  {"TAG": "2", "CLEAN": "Blood Pressure Measurement"},
  {"TAG": "3", "CLEAN": "Hypertensive Blood Pressure"},
  {"TAG": "4", "CLEAN": "Increased Hemoglobin A1C Level"},
  {"TAG": "5", "CLEAN": "Glycated Hemoglobin Measurement"},
  {"TAG": "6", "CLEAN": "Coronary Artery Disease"},
  {"TAG": "7", "CLEAN": "Mass"}
]
```

Input:
```
{note}
```

## Please respond with valid JSON only, no additional text. Output:
```

2.4. Assertion Status Classification

Purpose. Assigns clinical assertion categories (Present, Absent, Historical, Possible, Conditional, Hypothetical, Notassociated) to each entity.

```
## Determine the assertion status for every entity marked by <KEY> and </KEY> in
     this health record.

## Output a JSON list of dictionaries with:
- tag: entity order number (from KEY tags)
- assertion_status: one of Present, Absent, Historical, Possible, Conditional,
     Hypothetical, Notassociated

## Ambiguity guidelines:
- Prioritize most severe/current mention
- Medications: Present (active), Historical (discontinued), Conditional (PRN),
     Absent (refused/contraindicated), Possible (under consideration)
- "Patient at risk for X" = Hypothetical
- "No evidence of X" = Absent
- "Cannot rule out X" = Possible
- Family history -> Notassociated
- "Presenting with a history of X" or "X-day history of Y" -> treat Y as Present
     unless explicitly resolved

## Example:
Input:
```
```
```

```
CT showed <1>lesions</1> most likely secondary to <2>metastatic disease</2>. <3>
     Ativan</3> 0.5 mg IV q 4 to 6 hours prn <4>anxiety</4>. Patient presenting
     with a 7 day history of <5>headaches</5> and <6>fever</6>.
```

```
Output:
```
```
[
  {"tag": "1", "assertion_status": "Present"},
  {"tag": "2", "assertion_status": "Possible"},
  {"tag": "3", "assertion_status": "Present"},
  {"tag": "4", "assertion_status": "Conditional"},
  {"tag": "5", "assertion_status": "Present"},
  {"tag": "6", "assertion_status": "Present"}
]
```
```
Example (Medications):
Input: Patient was <1>prescribed metformin</1> but <2>refused due to insurance
     </2>. <3>Lisinopril</3> discontinued last month. <4>Tylenol</4> as needed
     for pain.

Output:
```
```
[
  {"tag": "1", "assertion_status": "Present"},
  {"tag": "2", "assertion_status": "Absent"},
  {"tag": "3", "assertion_status": "Historical"},
  {"tag": "4", "assertion_status": "Conditional"}
]
```
```

```
Input:
```
```
{note}
```
```

```
## Please respond with valid JSON only, no additional text. Output:
```

2.5. Attribute Enrichment

Purpose. Captures values, units, administration route, frequency, anatomical site, and descriptive modifiers for each entity.

```
## Extract detailed information for each marked entity in this health record.

## Output JSON as a list of dictionaries with these keys:
 - tag: entity's order number (matching the KEY number in the text)
 - body_location: anatomical location related to the entity (omit if not
     applicable)
 - value: numerical or text value(s) linked to the entity (use array for multiple
      values; exclude time information)
 - unit: units aligned with value (array if multiple)
 - infer: boolean indicating if each unit was inferred (array if multiple)
 - note: for numeric values, 'greater'/'lower'/'equal' comparator (array if
     multiple)
 - route: medication administration route (omit if not applicable)
```

```
 - freq: medication frequency (omit if not applicable)
 - other: descriptive modifiers not captured elsewhere

## Value extraction rules:
- Names/identifiers have no value
- Measurements and qualitative lab results have value
- Medication + dose: drug name has no value; dose is captured separately
- Use arrays when multiple values occur; arrays must align in length
- Time information belongs in "other", not in "value"

## Special handling:
- Panel tests: each value is a separate entity
- Infer units only when context is clear
- Measurements like "2 cm mass": value=2, unit=cm

## Examples:
Input:
```
<1>Metoprolol</1> 50 mg PO BID for <2>hypertension</2>. <3>Chem-7</3>:
     <4>127</4>, <5>3.6</5>, <6>88</6>.
```

Output:
```
[
  {"tag": "1", "value": "50", "unit": "mg", "infer": false, "note": "equal", "
       route": "PO", "freq": "BID", "other": "for hypertension"},
  {"tag": "2", "value": null},
  {"tag": "3", "value": null},
  {"tag": "4", "value": "127", "unit": "mEq/L", "infer": true, "note": "equal"},
  {"tag": "5", "value": "3.6", "unit": "mEq/L", "infer": true, "note": "equal"},
  {"tag": "6", "value": "88", "unit": "mEq/L", "infer": true, "note": "equal"}
]
```

Input:
```
<1>Glucose</1> readings: 112 mg/dL (fasting), 145 mg/dL (after meal). <2>Pain</2>
      in lower back rated 8/10 in morning, 6/10 in evening.
```

Output:
```
[
  {"tag": "1", "value": ["112", "145"], "unit": ["mg/dL", "mg/dL"], "infer": [
       false, false], "note": ["equal", "equal"], "other": "first reading was
       fasting, second was after meal"},
  {"tag": "2", "value": ["8/10", "6/10"], "body_location": "lower back", "other":
        "8/10 in morning, 6/10 in evening"}
]
```

Input:
```
<1>Irregular somewhat scalloped appearing hypoechoic focus</1> (2 cm) in left
     axillary tail. <2>Mass</2> measuring 3.5 x 2.8 cm noted in right breast.
```
```

```
Output:
```
[
  {"tag": "1", "value": "2", "unit": "cm", "body_location": "left axillary tail",
       "other": "Irregular somewhat scalloped appearing", "infer": false},
  {"tag": "2", "value": "3.5 x 2.8", "unit": "cm", "body_location": "right breast
      ", "infer": false}
]
```

```
Input:
```
<1>Blood pressure</1> elevated at 160/90 mmHg. <2>Temperature</2> normal, <3>
     pulse</3> irregular at 95 bpm.
```

```
Output:
```
[
  {"tag": "1", "value": ["elevated", "160/90"], "unit": [null, "mmHg"], "infer":
       [false, false], "note": [null, "equal"]},
  {"tag": "2", "value": "normal"},
  {"tag": "3", "value": "95", "unit": "bpm", "infer": false, "note": "equal", "
       other": "irregular"}
]
```

```
Input:
```
{note}
```

```
Please respond with valid JSON only, no additional text. Output:
```

2.6. Basic Patient and Visit Information

Purpose. Derives admission/discharge dates and demographics for temporal anchoring and schema export.

```
## Extract the following basic patient information from this health record:
 - admission_date
 - discharge_date
 - gender
 - death_date
 - birth_date
 - race
 - ethnicity
 - zip_code

Output as JSON Dictionary using the above as keys.
For dates, use "YYYY-MM-DD" format. If information is not explicitly mentioned,
     use null.

## IMPORTANT DATE EXTRACTION RULES:
- "Admission Date"/"Admitted on" -> admission_date
- "Discharge Date"/"Discharged on" -> discharge_date
```

```
- Visit/Encounter/Appointment or Date of Service -> use as both admission_date
    and discharge_date if same-day visit is likely
- "Date:", "Date of Note", provider-titled dates -> use as both admission_date
    and discharge_date
- Only one date present -> use it for both admission_date and discharge_date

Input:
```
{note}
```

Please respond with valid JSON only, no additional text. Output:
```

2.7. Per-Chunk Date Extraction (Single-chunk Context)

Purpose. Implements the date processing described in Step 3 of the Methods, assigning start and end dates to each entity using the current note as the temporal anchor.

```
## Extract dates for ALL entities marked by <KEY> tags in this health record.
    Every entity must have a date entry.

## Output JSON list of dictionaries with:
 - tag: entity order number from KEY tags
 - date: [YYYY-MM-DD, YYYY-MM-DD] for start and end (same date if single event;
     null if unknown)
 - inferred: [true/false, true/false] indicating whether each boundary was
     inferred

## Key Rules:
- Extract visit date as reference
- Explicit dates -> exact start/end, inferred=false
- "X days/weeks/months/years ago" -> END = visit date, START = visit date minus X
    , inferred=true
- Current symptoms/findings -> visit date for both boundaries, inferred=false
- New treatments/prescriptions -> START = visit date, END = null, inferred=[false
    , true]
- Chronic conditions without timeframe -> [null, null], inferred=[false, false]
- Related events share the main event's window
- Unknown timing -> [null, null], inferred=[false, false]

Input:
```
{note}
```

Please respond with valid JSON only, no additional text. Output:
```

2.8. Cross-Chunk Date Extraction (Multi-chunk Context)

Purpose. Continues the same date processing step when notes are chunked; uses the admission date anchor and the previous segment for disambiguation.

```
## Extract dates for ALL entities marked by <KEY> tags in this health record.
    Every entity must have a date entry.
```

```
## Output JSON list of dictionaries with:
 - tag: entity order number from KEY tags
 - date: [YYYY-MM-DD, YYYY-MM-DD] for start and end (same date if single event;
      null if unknown)
 - inferred: [true/false, true/false] indicating whether each boundary was
      inferred

## Key Rules:
- Use admission date as primary anchor
- Consider previous note but prioritize current segment
- Explicit dates -> exact start/end, inferred=false
- "X days/weeks/months/years ago" -> END = admission date, START = admission date
      minus X, inferred=true
- Current symptoms/findings -> admission date for both boundaries, inferred=false
- New treatments/prescriptions -> START = admission date, END = null, inferred=[
     false, true]
- Chronic conditions without timeframe -> [null, null], inferred=[false, false]
- Related events share the main event's window
- Unknown timing -> [null, null], inferred=[false, false]

Input:
```
Previous piece of the record (context):
{prev_note}
Admission date: {adm_date}, Discharge date: {dis_date}
Current note:
{note}
```

Please respond with valid JSON only, no additional text. Output:
```

2.9. Admission and Discharge Date Finder

Purpose. Supports the metadata retention described in Step 1 (chunking) by recovering visit anchors when only coarse dating cues are present.

```
## You will receive an electronic health record for a patient. Your task is
     extract the admission date and the discharge date of the patient.
## Your output should be in JSON Array format: "[admission date, discharge date
     ]". If this does not apply to the record provided, for example, use null to
     fill up the value (such as [null, null]).

Input:
{note}

Please respond with valid JSON only, no additional text. Output:
```

2.10. Date Normalization (Helper within Step 3)

Purpose. Auxiliary helper used inside the date processing step to normalize relative expressions to absolute start–end ranges using an anchor date; not an additional pipeline stage.

```
## Given an anchor time in the form of YYYY-MM-DD, what is the date range of "{
     date}" when the anchor time is {anchor}?
## Your answer should be given in the form of [YYYY-MM-DD, YYYY-MM-DD]
     representing the start and end time.
```

```
Please respond with valid JSON only, no additional text. Output:
```

2.11. JSON Repair (Robustness Helper)

Purpose. Internal robustness helper that auto-corrects malformed JSON from LLM calls; it does not introduce a new pipeline step and only safeguards existing outputs.

```
## I have a JSON file with errors that cannot be parsed. When I tried to parse it
      using `demjson3.decode`, I received specific error messages. Please correct
      the JSON and respond with only the corrected JSON content--do not include
     any explanations, comments, or additional text.

Error: {error_message}
Input JSON:
{json_content}

## Please respond with only the corrected JSON. Output:
```

3. JSON Output Specification

The pipeline aggregates module outputs into a consolidated JSON structure before any format conversion. Field names are presented in monospace to distinguish schema tokens from prose. Missing or inapplicable entries are set to `null`.

Field	Type	Definition
`term_index`	integer	Position of the entity in the note after ordering.
`key`	string	Note or encounter identifier provided to the system.
`admission_date`, `discharge_date`	string (YYYY-MM-DD)	Visit-level anchors propagated to each entity.
`gender`, `death_date`, `birth_date`, `race`, `ethnicity`, `zip_code`	string	Demographic attributes extracted once per note.
`mention`	string	Original entity span text.
`context`	string	Surrounding phrase or sentence for auditability.
`mention_start_pos`, `mention_end_pos`	integer	Character offsets within the original note (start inclusive, end exclusive).

Field	Type	Definition
code	string	Ontology identifier concatenated with preferred term as code||preferred_term. Derived via SapBERT retrieval over UMLS.
type	string	UMLS semantic type associated with code.
assertion_status	categorical	One of Present, Absent, Historical, Possible, Conditional, Hypothetical, Notassociated.
body_location	string	Anatomical site text.
body_location_code	string	Ontology mapping for body_location using the same concatenation convention as code.
value	string or array	Numeric or qualitative measurement linked to the entity; arrays hold parallel multi-value entries.
unit	string or array	Measurement unit(s) aligned with value.
infer	boolean or array	Indicates whether each unit was inferred rather than explicit.
note	string or array	Comparator for numeric values (greater, lower, equal) or brief qualifier.
freq	string	Medication frequency phrase.
route	string	Medication administration route.
other	string	Additional descriptors not captured elsewhere (for example "fasting", "irregular").
related	object	Directed relations to other entities; keys are target tags and values are relation types.
begin_date, end_date	string (YYYY-MM-DD)	Normalized start and end dates for the entity.
begin_date_inferred, end_date_inferred	boolean	Flags indicating whether each date boundary was inferred.

4. i2b2 Export Schema

Structured outputs are rendered to the i2b2 OBSERVATION_FACT layout with lightweight extensions for clinical NLP. Each entity produces one base row plus optional modifier rows when additional attributes are present. Field names follow the canonical i2b2 spelling to preserve compatibility.

4.1. Field Definitions

Field	Type	Definition
patient_num	integer	Pseudonymized patient identifier (provided upstream).

Field	Type	Definition
encounter_num	integer or string	Visit identifier; equals the pipeline key for the note.
birth_date, death_date	date	Demographic dates when present.
sex_cd, race_cd, ethnicity_cd, zip_cd	string	Demographic codes or free-text values from the note.
start_date, end_date	date	Entity time span; defaults to visit anchors when explicit timing is absent.
concept_cd	string	Primary concept code from UMLS mapping (or downstream conversion).
name_char	string	Human-readable mention text.
observation_blob	string	Local context window for audit.
modifier_cd	string	Modifier type; @ for base record, otherwise codes such as DOSE, ROUTE, FREQ, ASSERTION_STATUS, BODY_LOCATION, OTHER_INFO, RELATED.
instance_num	integer	Sequential counter to distinguish multiple modifiers attached to the same entity.
valtype_cd	categorical	N for numeric, T for text.
tval_char	string	Text value or comparator when valtype_cd is text or when expressing greater/lower/equal.
nval_num	numeric	Numeric value when applicable.
valueflag_cd	categorical	Abnormality flags if present (High, Low, Abnormal); otherwise null.
units_cd	string	Measurement unit.
unitflag_cd	boolean (string)	Indicates whether the unit was inferred (true/false).
entity_index	integer	Position of the entity in the original note (copied from term_index).
code_type	string	Coding system label (ICD10CM, LNC, RXNORM, SNOMEDCT_US, or CUI fallback).

4.2. Base and Modifier Records

- **Base record** (modifier_cd = @): carries the primary concept, context, dates, and any value-unit pair directly associated with the entity.
- **Dose, route, frequency** modifiers: emitted when value, route, or freq appear in the JSON output.
- **Assertion status and body location** modifiers: capture clinical status and anatomical site for downstream phenotyping.

- **Other information** modifier: holds descriptive qualifiers not mapped elsewhere.
- **Related** modifier: serializes the directed relationship dictionary to maintain cross-entity links.

5. UMLS Retrieval Procedure

The concept enrichment module aligns cleaned entity mentions to UMLS concepts using SapBERT embeddings (`cambridgeltl/SapBERT-from-PubMedBERT-fulltext`) with FAISS nearest-neighbour search under an inner-product metric on L_2-normalized vectors (equivalent to cosine similarity). Two independent indices are built: an *all-terms* index over the full deduplicated UMLS term list, and a restricted *body-location* index used for anatomical-site resolution. The index type is selected by corpus size: for term lists larger than 50,000 entries an inverted-file index (`IndexIVFFlat`) with an `IndexFlatIP` coarse quantizer is used, with the number of Voronoi cells set to nlist $= \min(4096, \max(\lfloor N/50 \rfloor, 256))$ and query breadth nprobe $= \min(32, \text{nlist})$; for smaller lists an exact `IndexFlatIP` index is used. The pseudocode below mirrors the deployed implementation.

Algorithm 1 UMLS Concept Retrieval with SapBERT and FAISS

1: Load the UMLS term list with concept unique identifiers (CUIs) and semantic types; deduplicate while preserving term–CUI pairs.
2: Encode all terms with the SapBERT encoder; apply L_2 normalization to obtain unit vectors of dimension d.
3: **Index construction** (per index: all-terms, body-location):

1. If $N > 50{,}000$: set nlist $= \min(4096, \max(\lfloor N/50 \rfloor, 256))$; build `IndexIVFFlat` with an `IndexFlatIP` quantizer and `METRIC_INNER_PRODUCT`; train on the embeddings (subsampling to at most 10^6 vectors when $N > 10^6$); add all vectors.
2. Else: build an exact `IndexFlatIP` index and add all vectors.
3. Attempt GPU construction (`StandardGpuResources`); on GPU failure, retry the same construction on CPU.

4: **Query** (batched): filter out empty or null mentions; encode remaining mentions with the same SapBERT encoder and L_2 normalization; cast to `float32`.
5: For `IndexIVFFlat`, set nprobe $= \min(32, \text{nlist})$; search the appropriate index for the top-k matches ($k = 1$ by default).
6: For each mention, return a JSON object mapping the retrieved CUI to its preferred term and semantic type; attach the retrieved codes to the corresponding entities.

Body-location mentions are additionally resolved against the body-location index so that anatomical sites map to anatomy-typed concepts rather than to the nearest concept of any semantic type. The retrieval is deterministic given the encoder, the term list, and the index parameters above; no external network calls are made during concept mapping.

6. Mapping UMLS Concepts to Source Terminologies

CLINES normalizes each extracted mention to a UMLS Concept Unique Identifier (CUI) using the SapBERT + FAISS retrieval procedure of the preceding section. The CUI is the interoperability hub: in the UMLS Metathesaurus a single CUI aggregates synonymous terms contributed by many independently maintained source vocabularies, so a CUI can be deterministically crosswalked to any of those vocabularies through the standard `MRCONSO` concept–atom table without further model inference. The i2b2-style export therefore stores, for every entity, the CUI, its UMLS preferred term, and its semantic type (`MRSTY`); downstream consumers obtain a code in their terminology of choice by joining on the CUI.

Downstream use	Target vocabulary (UMLS source)	Example crosswalk from CUI
Problem lists / diagnoses	SNOMED CT (`SNOMEDCT_US`)	C0020538 → 38341003 (Hypertensive disorder)
Billing / claims	ICD-10-CM (`ICD10CM`)	C0020538 → I10 (Essential hypertension)
Medications	RxNorm (`RXNORM`)	C0025598 → 6809 (Metformin)
Laboratory tests	LOINC (`LNC`)	C0202194 → 4548-4 (Hemoglobin A1c)
Procedures	CPT / SNOMED CT procedure	C0009378 → 44970 (Appendectomy)

Table 3: Because every entity is anchored to a UMLS CUI, a single normalization step yields codes in arbitrarily many downstream terminologies via standard UMLS crosswalks, without re-running the model. The example crosswalks are illustrative of widely used UMLS source mappings; the specific target codes available for a given CUI depend on the licensed UMLS source vocabularies.

This design decouples extraction from any single coding standard: institutions that require SNOMED CT for clinical documentation, ICD-10-CM for reimbursement, or RxNorm for medication reconciliation all consume the same CUI-anchored output, and the mapping layer is auditable because each CUI carries its preferred term and semantic type alongside the source-vocabulary codes.

7. Detailed Performance Metrics with Bootstrap Confidence Intervals

This section expands Figure 5a of the main text with the full per-field, per-dataset breakdown for the four full-pipeline backbones evaluated under the identical CLINES workflow. For every field we report the point F1 with a bias-corrected 95% bootstrap confidence interval (2,000 resamples over notes; seed 20260525). *Code* F1 requires both correct span detection and correct UMLS

concept assignment; *Assertion*, *Value*, and *Unit* are conditional on a correctly matched mention; *Begin/End date* F1 reflect temporal-boundary agreement after relative-date normalization. The mention-level breakdown and single-prompt / deep-learning baselines appear in Figure 5d.

Backbone	Dataset	Code	Assertion	Value	Unit	Begin date	End date
o3-mini-medium	4CE	0.874 (.849–.896)	0.884 (.857–.909)	0.815 (.748–.867)	0.771 (.710–.826)	0.555 (.495–.624)	0.560 (.498–.637)
	CORAL-B	0.814 (.787–.838)	0.840 (.803–.871)	0.801 (.711–.858)	0.752 (.675–.810)	0.683 (.614–.756)	0.690 (.622–.764)
	CORAL-P	0.848 (.834–.865)	0.873 (.857–.887)	0.905 (.877–.932)	0.895 (.863–.926)	0.708 (.630–.783)	0.722 (.638–.797)
GPT-4o	4CE	0.781 (.759–.801)	0.842 (.825–.858)	0.771 (.715–.822)	0.764 (.698–.823)	0.780 (.627–.867)	0.729 (.443–.847)
	CORAL-B	0.701 (.667–.731)	0.761 (.714–.799)	0.617 (.546–.683)	0.545 (.476–.612)	0.734 (.636–.820)	0.735 (.634–.821)
	CORAL-P	0.781 (.751–.810)	0.852 (.831–.872)	0.857 (.808–.900)	0.761 (.687–.840)	0.780 (.706–.843)	0.772 (.698–.836)
Llama-3.1-405B	4CE	0.665 (.641–.690)	0.738 (.715–.760)	0.711 (.660–.761)	0.655 (.588–.713)	0.706 (.573–.767)	0.608 (.208–.715)
	CORAL-B	0.608 (.577–.636)	0.706 (.672–.733)	0.667 (.590–.734)	0.735 (.671–.791)	0.653 (.600–.714)	0.630 (.542–.713)
	CORAL-P	0.655 (.624–.686)	0.746 (.722–.770)	0.776 (.705–.836)	0.720 (.649–.782)	0.629 (.540–.706)	0.635 (.550–.707)

Table 4: Per-field F1 with 95% bootstrap confidence intervals for the four full-pipeline backbones across the three expert-annotated datasets (4CE; CORAL breast cancer, CORAL-B; CORAL pancreatic cancer, CORAL-P). CORAL-P value/unit fields exceed code F1 because numeric laboratory and pathology measurements are densely and explicitly reported in oncology synoptic notes. Date-field F1 is uniformly lower, reflecting the difficulty of resolving relative and implicit temporal expressions.

8. Inter-Annotator Agreement: Per-Note Detail

This section provides the per-note inter-annotator agreement (IAA) results underlying Figure 4a and the headline agreement statistics in the main text. For the cross-annotation study, three notes per dataset (4CE; CORAL) were independently re-annotated by a second annotator, blinded to the original assignment.

We report a different metric for each field type, matching field semantics to a well-established IAA convention:

- **Mention** (open span detection): **set-based F1**. For set-based agreement on an open candidate space, F1 is the appropriate measure—Cohen's κ requires a closed negative class that is ill-defined here, and is therefore not reported for mention-level agreement [**?**]. We additionally report the Jaccard index for transparency.
- **Assertion** (closed 5-class categorical): **Cohen's** κ computed over the kept-by-both subset, after collapsing the 8 raw labels to the standard 5 classes (Present / Absent / Possible / Conditional / Notassociated).
- **Value, Unit, Begin date, End date** (free-text fields with substantial "N/A"-by-design coverage): **exact-match rate over the kept-by-both subset, counting both annotators leaving the field empty as agreement**. This convention reflects that for many mentions the field is genuinely not applicable (e.g. "hypertension" has no numeric value), and both annotators correctly leaving the field empty is itself an annotation-quality signal. The metric thereby penalizes over-annotation symmetrically with mis-extraction.

Type (UMLS semantic type) is omitted because in our pipeline this field is assigned by the SapBERT normalization step rather than independently labeled by annotators; annotator agreement on this field reflects acceptance of the model's automatic type label and is not informative about human inter-rater reliability.

Dataset	Note	n items	n kept-by-both	Mention F1	Assertion κ	Value	Unit	Begin date	End date
4CE	report03	335	237	0.889	0.842	0.717	0.966	0.717	0.734
4CE	report04	302	184	0.807	0.860	0.717	0.984	0.397	0.402
4CE	KUMC_7	375	145	0.662	0.773	0.669	0.972	0.497	0.483
CORAL	pdac_17	446	254	0.901	0.664	0.839	0.988	0.913	0.906
CORAL	pdac_7	559	358	0.845	0.940	0.662	0.989	0.891	0.925
CORAL	pdac_14	658	340	0.785	0.900	0.679	0.974	0.935	0.859
4CE pooled		1,012	566	0.793	0.828	0.705	0.973	0.557	0.562
CORAL pooled		1,663	952	0.836	0.857	0.715	0.983	0.913	0.896
Pooled		2,675	1,518	**0.820**	**0.848**	**0.711**	**0.980**	**0.780**	**0.771**

Table 5: Per-note IAA on the cross-annotation subset (three notes per dataset; 4CE and CORAL). Mention agreement is reported as set-based F1 (per Hripcsak & Rothschild 2005 [**?**], F1 is the appropriate IAA measure when the negative class is unbounded; Cohen's κ is mathematically ill-defined in this setting and is therefore omitted). Assertion is reported as Cohen's κ on the 5-class collapsed label space (Present / Absent / Possible / Conditional / Notassociated) computed over the kept-by-both subset. Value, Unit, and Date fields are reported as exact-match rate over the kept-by-both subset, counting both annotators leaving a field empty as agreement; this convention rewards correctly recognizing that a field is not applicable and penalizes over-annotation. The marked 4CE–CORAL gap on date agreement (0.557–0.562 vs. 0.913–0.925) reflects that 4CE comprises admission notes in which most mentions inherit dates implicitly from the note-level admission date—a setting where annotators can legitimately disagree about whether to infer the date or leave the field empty—whereas CORAL longitudinal oncology notes carry explicit per-mention dates. The pooled mention F1 (0.820) falls within the per-pair F1 range reported for the i2b2 2010 clinical-NER challenge (0.78–0.91; Uzuner et al. 2011 [**?**]), and the pooled assertion κ (0.848) is in the substantial range (Landis & Koch 1977 [**?**]).

9. Hallucination Taxonomy and Case Studies

To characterize the nature of CLINES' apparent false positives (FPs) against the gold standard, we drew a stratified sample of candidate FP predictions and had each manually adjudicated into a seven-class taxonomy by a reviewer with access to the full source note. Because the sample was stratified by an automatic rule-based pre-classifier, we extrapolate the per-stratum judge distribution to the full FP population ($N = 7{,}171$) by post-stratified Horvitz–Thompson estimation: within each rule stratum the judged category proportions are scaled to that stratum's population size and summed across strata. The seven classes distinguish genuine model errors (fabricated entity, span-boundary error, wrong assertion, wrong code, wrong value, wrong date) from "not an error", where the prediction is in fact defensible and the discrepancy reflects a gap or inconsistency in the reference annotation rather than a model hallucination.

Category	Extrapolated count	% of candidate FPs
Not an error (annotation gap)	3,895	54.31
Wrong code	1,199	16.72
Wrong assertion	621	8.67
Wrong value	596	8.31
Fabricated entity	388	5.41
Span-boundary error	387	5.39
Wrong date	85	1.19
True hallucination (6 categories)	**3,276**	**45.69**
Total candidate FPs	**7,171**	**100.0**

Table 6: Seven-class hallucination taxonomy over the full candidate-FP population, extrapolated from $n = 700$ manually judged predictions via post-stratified Horvitz–Thompson estimation. The single largest category is "Not an error": a majority (54.3%) of apparent FPs are defensible predictions that the reference annotation omitted or recorded inconsistently. No individual *true*-hallucination category exceeds 17% of FPs, and outright "Fabricated entity" errors account for only 5.4%.

9.1. Representative case studies

The following examples are drawn verbatim (mentions and adjudication rationales) from the manual-judging study and illustrate each major category.

Not an error (annotation gap).
CLINES extracted *"RLS"* and normalized it to *restless legs syndrome* (C0035258). The abbreviation and its expansion are explicitly present in the note's problem list, but the rule-based reference annotation did not capture it. The prediction is correct; the discrepancy is a gap in the gold standard.

Wrong code.
For *"HR+/HER2- breast cancers"* CLINES assigned C3539878 (*ER-negative, PR-negative, HER2-negative breast cancer*). The surrounding text describes a hormone-receptor-*positive*, HER2-negative tumor, so the assigned concept inverts the biomarker status—a genuine normalization error.

Wrong assertion.
"Penicillins" (C0030842) was marked Present, but the note states the patient's penicillin-allergy status is uncertain ("not sure if she is allergic"), which the schema treats as Conditional. The concept is correct; the assertion class is wrong.

Span-boundary error.
CLINES extracted *"TYLENOL"* where the gold span is *"acetaminophen (TYLENOL)"*. The normalized concept and code (C0699142) are identical and correct; only the surface span is truncated.

Fabricated entity.
"Age of Onset" was extracted (and coded to C3175531) from a *column header* in a family-history table rather than from any patient-specific clinical

statement—an entity that does not correspond to a real clinical concept in context.

These case studies motivate the main-text argument that raw precision against a single-pass reference annotation systematically understates CLINES' true reliability, because the dominant "error" mode is recovery of clinically valid mentions the reference omitted, not fabrication.

10. Output Consistency Across Repeated Runs

Large-language-model outputs can vary across repeated invocations even at low decoding temperature, owing to non-deterministic kernel scheduling and floating-point non-associativity on accelerator hardware. Because CLINES is intended for chart-review-like tasks where reproducibility matters, we assess run-to-run stability of the end-to-end pipeline with a pre-specified protocol.

10.1. Protocol

We select a fixed subset of 20 notes (balanced across the 4CE and CORAL datasets) and run the complete CLINES pipeline $R = 5$ times independently on each note with identical inputs and configuration (`gpt-4o-mini` backbone; temperature as deployed; distinct random seeds where exposed by the API). Mention identity is defined *semantically*, not by exact span string: two extracted mentions are considered equivalent if they share a non-empty UMLS code or share a canonical surface form (lowercased, punctuation-stripped, whitespace-collapsed). This absorbs span-boundary jitter and surface synonyms that the pipeline mapped to the same concept, both of which exact-span matching would otherwise count as disagreement. For each note we compare the five extracted i2b2 record sets and quantify stability with:

- *Pairwise mention-set Jaccard* (over semantic equivalence-class ids) and *code-set Jaccard* across the $\binom{5}{2} = 10$ run pairs, summarizing the overlap of extracted mentions and assigned UMLS codes;
- *Per-field majority-vote stability*: for every equivalence class that appears in at least two runs, the fraction of those runs whose value for a given field (assertion status, value, unit, begin date, end date) equals the modal value across runs; averaged over all such (class, field) cells. When a class fires multiple times within one run, that run's representative value for the cell is the run's own modal value for the class.

This protocol mirrors the output-consistency assessment recommended by Ntinopoulos et al. (BMJ Health & Care Informatics 2025) for clinical LLM evaluations, which the present study did not originally include.

10.2. Results

All 20 notes produced complete outputs in all five runs (no run failures; no partial outputs). Across the $\binom{5}{2} = 10$ run pairs per note, the mean semantic mention-set Jaccard was 0.752 (per-note range 0.664–0.861) and the mean code-set Jaccard was 0.622 (range 0.498–0.718); the gap of ≈ 0.13 between mention-set and code-set Jaccard reflects mentions for which the pipeline assigned no UMLS code (rather than disagreement at the concept level). Approximately three quarters of extracted mentions and roughly 62% of UMLS codes are therefore shared across any two runs on the same note, consistent with stochastic LLM decoding at non-zero temperature.

For mentions that *are* recovered across runs, the structured fields associated with each mention are highly stable: averaging over all 27,605 (class, field) cells with at least two contributing runs, the modal field value was matched in 89.9% of contributing runs on average (Table 7). Field-level stability was highest for unit (94.9%), end date (94.2%), and begin date (91.3%), and slightly lower for assertion status (84.9%) and numeric value (84.3%). Together these two views indicate that, although the LLM does not deterministically reproduce the full extracted set on every run, the per-field content of each extracted mention is largely run-invariant; in chart-review use, an analyst aggregating multiple runs (e.g., via majority vote over $R \geq 3$ runs) recovers a stable consensus output.

Table 7: Per-field majority-vote stability across $R = 5$ independent runs on the 20-note subset (mean fraction of contributing runs equal to the modal value; n_{cells} is the number of (mention-class, field) cells with ≥ 2 contributing runs, using semantic mention identity).

Field	**Mean modal agreement**	**Per-cell range**	$\boldsymbol{n_{\text{cells}}}$
Assertion status	0.849	[0.250, 1.000]	5,521
Numeric value	0.843	[0.200, 1.000]	5,521
Unit	0.949	[0.200, 1.000]	5,521
Begin date	0.913	[0.200, 1.000]	5,521
End date	0.942	[0.200, 1.000]	5,521
All fields pooled	**0.899**	[0.200, 1.000]	27,605

11. Reporting Checklists: MI-CLAIM and TRIPOD+AI

Because CLINES is a clinical artificial-intelligence system rather than an observational epidemiological study, we report it against AI-specific reporting standards—the Minimum Information about Clinical Artificial Intelligence Modeling (MI-CLAIM) checklist and the TRIPOD+AI statement—in place of the STROBE checklist used for observational studies. The tables below map each checklist's principal items to where they are addressed in the manuscript and supplement.

MI-CLAIM item	Where addressed
1. Study design, clinical problem, cohort/data source, ground truth	Methods (study design, datasets, annotation protocol); Ethics Approval
2. Data preparation, partitioning, and pre-processing	Methods (chunking and pre-processing); Supplementary prompt suite
3. Model description and optimization (architecture, training-free use of LLMs)	Methods (pipeline stages); Supplementary prompt suite and retrieval procedure
4. Model performance, evaluation metrics, uncertainty	Results; Figures 3–5; Supplementary detailed-metrics and IAA tables
5. Model examination (error analysis, ablation, robustness)	Results (ablation, hallucination analysis, note-length); Figures 4–5; Supplementary hallucination and consistency sections
6. Reproducibility (code, prompts, models, data access tier)	Data Sharing (MIT-licensed reference implementation); Supplementary prompt suite, schemas, and retrieval pseudocode

Table 8: MI-CLAIM (Norgeot et al., Nat Med 2020) item-to-location mapping. CLINES is used in a training-free configuration, so MI-CLAIM optimization items pertaining to model training are addressed by the prompt-suite and retrieval specifications rather than by a training procedure.

TRIPOD+AI item (grouped)	Where addressed
Title and structured abstract	Title; Abstract (Objective / Methods and Analysis / Results / Conclusion)
Background and objectives	Introduction
Data sources, settings, participants	Methods (datasets); Ethics Approval; Data Sharing
Predictors/inputs, outcome/target, ground-truth definition	Methods (task definition, i2b2 schema, annotation)
Sample size and missing data	Methods; Results (per-dataset note and mention counts)
AI model and analytical methods	Methods (pipeline); Supplementary prompt suite and retrieval procedure
Model performance and uncertainty quantification	Results; Figures 3–5; Supplementary detailed-metrics (bootstrap 95% CIs)
Model evaluation, comparison with baselines, ablation	Results; Figure 5; Supplementary tables
Limitations and generalizability	Discussion (Limitations; Dataset-level variation)
Funding, conflicts, data and code availability	Funding; Declaration of Interests; Data Sharing

Table 9: TRIPOD+AI (Collins et al., BMJ 2024) item-to-location mapping, grouped by checklist domain. Items specific to incremental-value and calibration analyses for risk-prediction models are not applicable to an information-extraction system and are reported as not applicable.